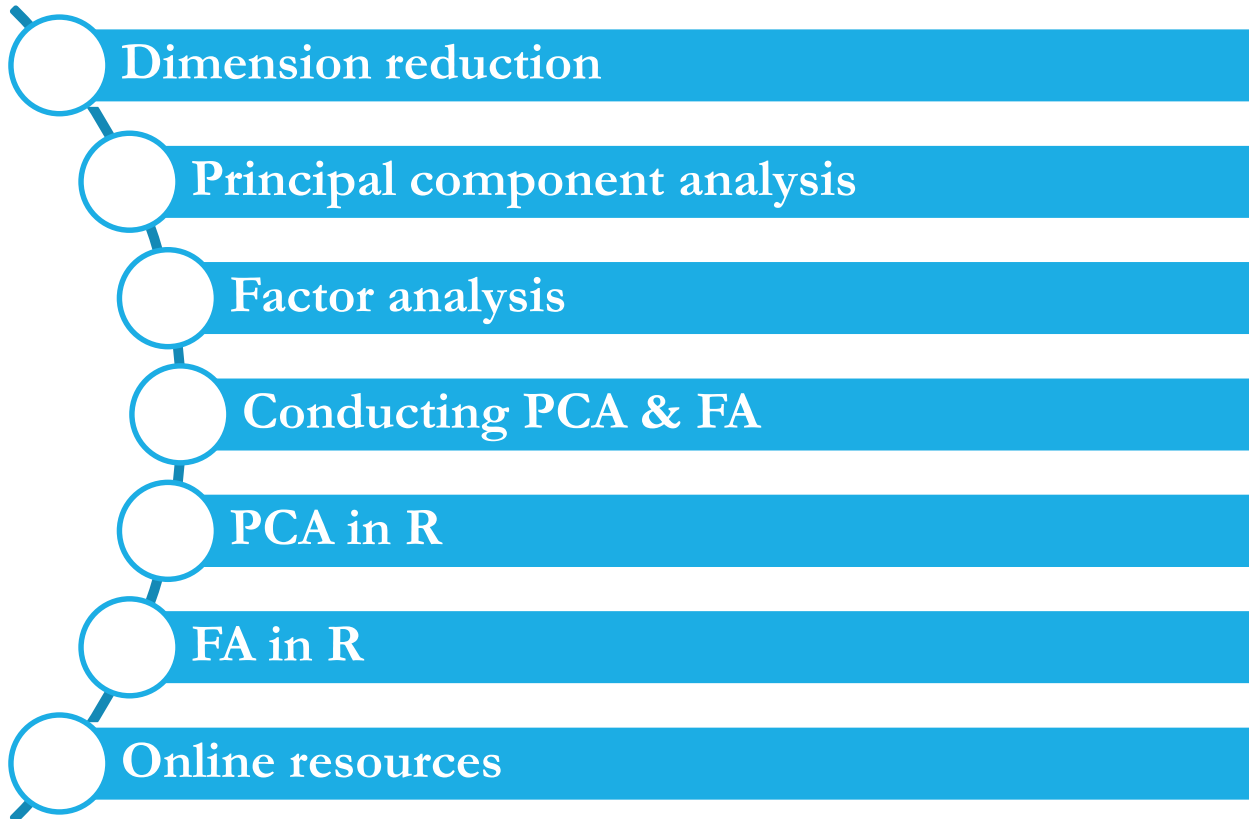


Dimension reduction



Dimension reduction

- Unlike variables directly measured such as speed, height, weight, etc., some variables such as egoism, creativity, happiness, religiosity, comfort are not a **single measurable entity**
- They are **constructs** that are derived from the measurement of other, directly observable variables
- Constructs are usually defined as unobservable latent variables which may include several **dimensions**.



Is used to:

- identify and describe “dimensions” that underlie the input data
- reduce the dimensionality of the research problem
 - benefit = simplification; reduce number of variables you have to worry about
- identifying sets of variables with similar “behaviour”



- It is more art than science:
 - There is a number of extraction methods (PCA, FA, etc.)
 - There is a number of rotation methods (Orthogonal, Oblique, Varimax, etc.)
 - How to choose the number of factors to extract?
 - How to estimate communalities?
 - How to interpret the factors/components?
 - ...

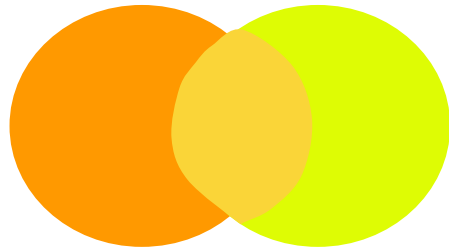


- We can do anything we want to the numbers, provided we **don't distort** the spatial relationships that exist among cases
- Some kinds of manipulations help us think about the **shape** of the scatter in more productive ways
- Most common manipulations are **extraction and rotation**
- Why bother? – they provide **more “efficient” description** of the observed phenomenon

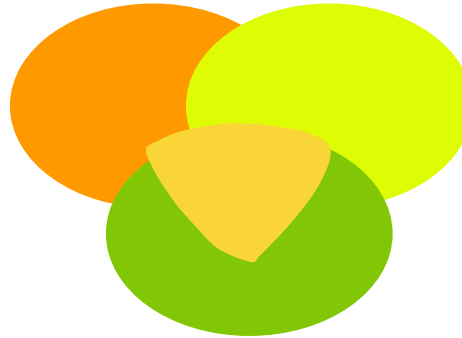


Basic idea - Extraction

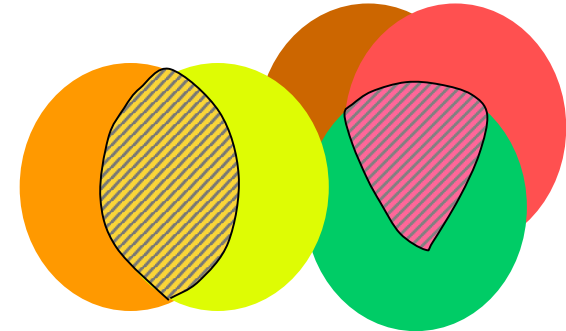
Two variables (A, B)



Three variables (A, B & C)



Multiple variables
(A, B, C, D, & E)



Refers to common variance or extracted factor



Refers to common variance or extracted factor 1



Refers to common variance or extracted factor 2



- Discover a new set of factors/dimensions/axes against which to represent, describe or evaluate the data
 - For more effective reasoning, insights, or better visualization
 - Reduce noise in the data
 - Typically a smaller set of factors: dimension reduction
 - Can build more effective data analyses on the reduced-dimensional space: classification, clustering, pattern recognition
- Factors are combinations of observed variables
 - Observed data are described in terms of these factors rather than in terms of original variables/dimensions



Why Factor or Principal component analysis?

- We study phenomena that **can not be directly observed**
- We want to **identify** and operate with underlying latent factors rather than the observed data
- We want to **discover and exploit** hidden relationships
- We have **too many observations** and dimensions
- Combinations of observed variables may be **more effective** bases for insights, even if physical meaning is obscure

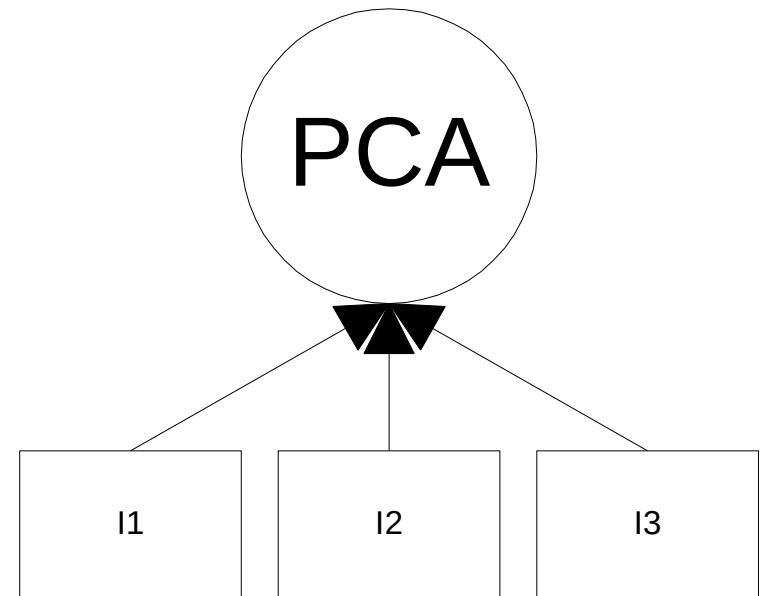
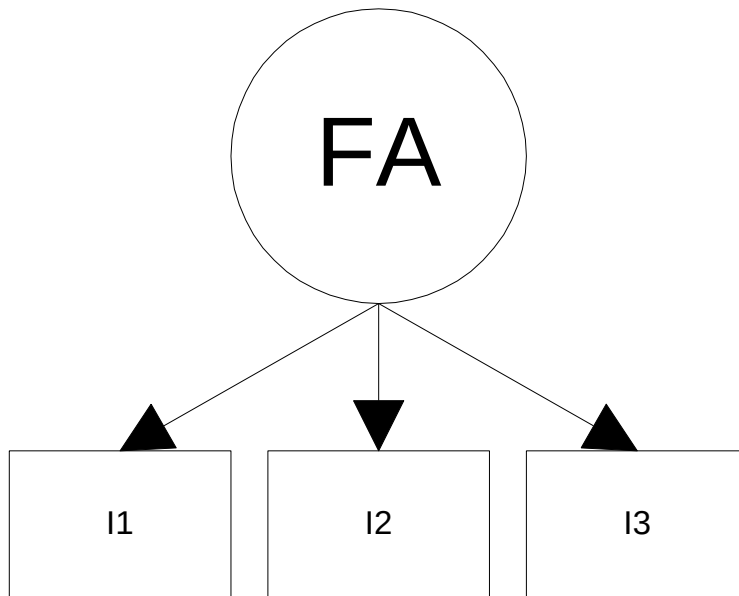


FA vs. PCA conceptually

- **FA** analyzes only the **variance shared** among the variables
(common variance without error or unique variance)
PCA analyzes **all of the variance**
- **FA**: “What are the **underlying processes** that could produce these correlations?”
PCA: Just **summarize empirical associations**, very data driven



Factor analysis vs. Principal component analysis



- **Principal component analysis** is done on **covariance matrix**
 - Variables must be in same units
 - Emphasizes variables with most variance
 - Mean eigenvalue $\neq 1.0$
- **Factor analysis** is done on **correlation matrix**
 - Variables are standardized (mean 0.0, SD 1.0)
 - Variables can be in different units
 - All variables have same impact on analysis
 - Mean eigenvalue = 1.0



Principal component analysis (PCA)

- Principal component analysis (PCA) is a way to reduce data dimensionality
- PCA projects high dimensional data to a lower dimension
- The new variables/dimensions are linear and uncorrelated combinations of the original ones
- PCA projects the data in the least square sense– it captures big (principal) variability in the data and ignores small variability

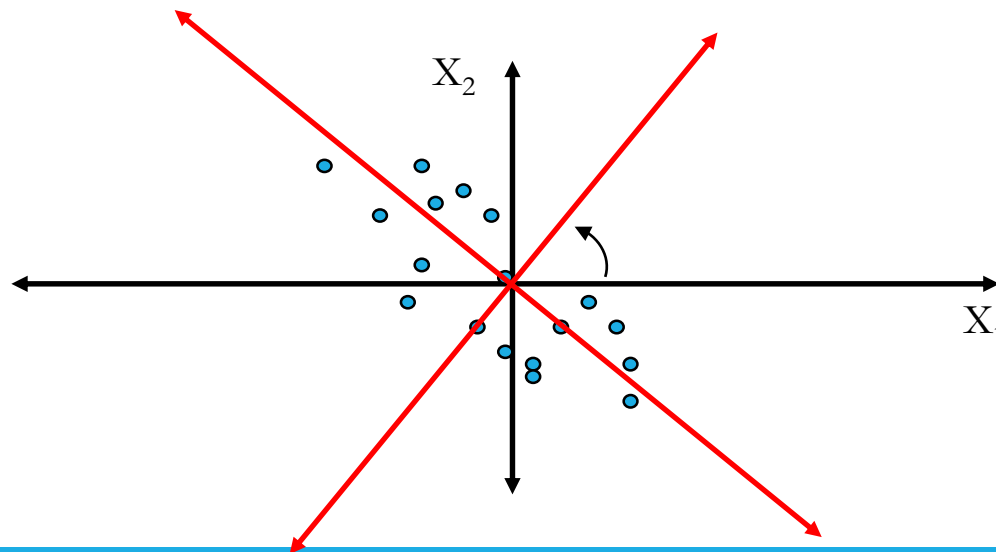


When is using PCA appropriate?

- When your data is interval or ratio level
- When you have at least 5 observations per variable and at least 100 observations (ie...20 variables > 100 observations)
- When significant correlations exist among variables
- When trying to reduce the number of variables to be used in another GLM technique (regression, MANOVA, etc...)
- When attempting to identify latent constructs that are being measured by observed variables



Suppose we have a population measured on p random variables $X_1, \dots, X_p$. Note that these random variables represent the p -axes of the coordinate system in which the population resides. Our goal is to develop a new set of p axes (linear combinations of the original p axes) in the directions of greatest variability:



From k original variables: $x_1, x_2, \dots, x_k$:

Produce k new variables: $y_1, y_2, \dots, y_k$:

$$y_1 = a_{11}x_1 + a_{12}x_2 + \dots + a_{1k}x_k$$

$$y_2 = a_{21}x_1 + a_{22}x_2 + \dots + a_{2k}x_k$$

...

$$y_k = a_{k1}x_1 + a_{k2}x_2 + \dots + a_{kk}x_k$$

such that:

y_k 's are **uncorrelated** (orthogonal)

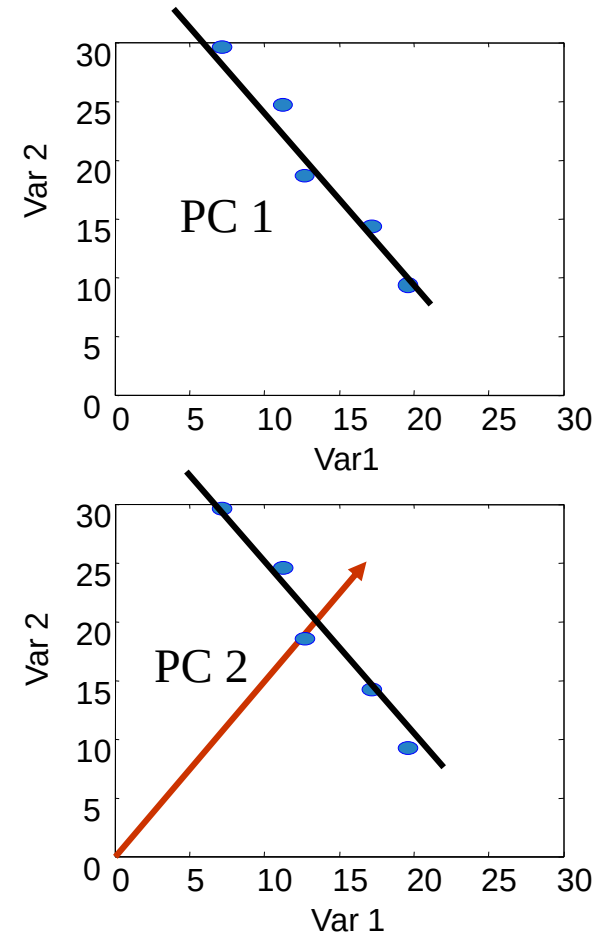
y_1 explains as much as possible of original variance in data set

y_2 explains as much as possible of remaining variance

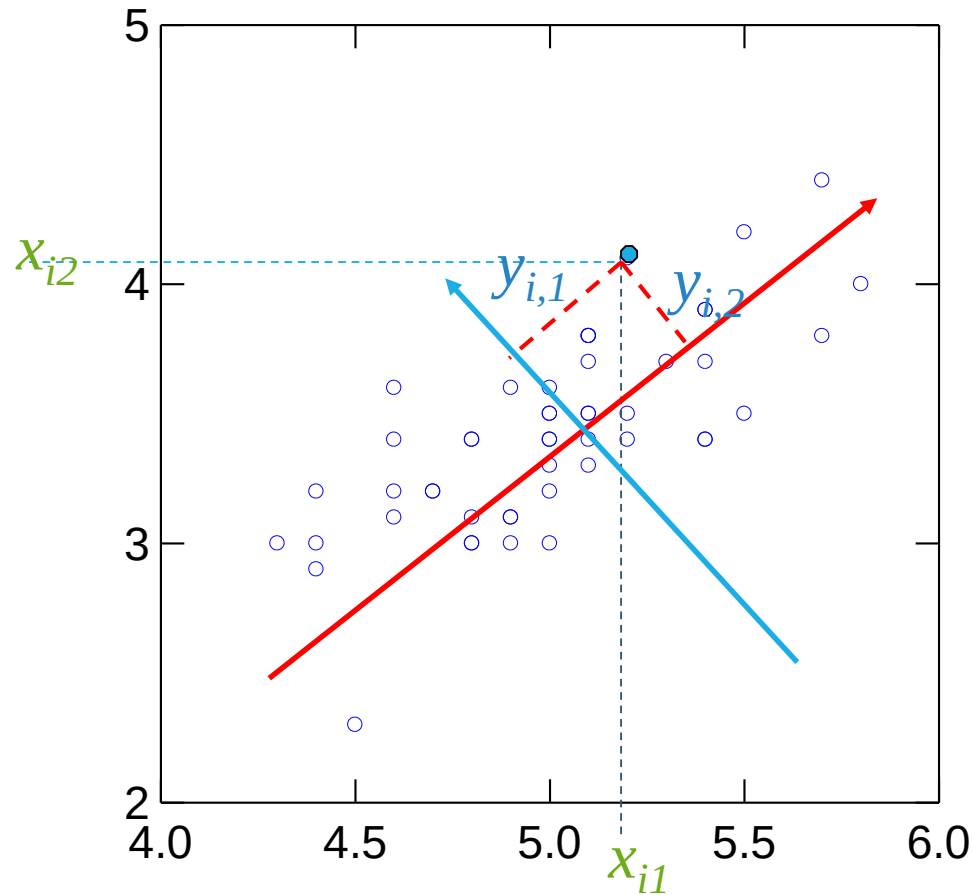
etc.



- All principal components (PCs) start at the origin of the ordinate axes.
- First PC is direction of **maximum variance** from origin
- Subsequent PCs are orthogonal to 1st PC and describe maximum residual variance



PCA Scores



- PCA:
 - Finds orthonormal basis for data
 - Sorts dimensions in order of “importance”
 - Discards low significance dimensions
- Is not magic:
 - Doesn’t know class labels
 - Can only capture linear variations
- One of many tricks to reduce dimensionality!



1. Procedure to conduct PCA/Factor analysis?
2. How many components/factors to retain?
3. Which type of rotation to use?
4. How to interpret the output?
5. How to interpret the retained components/factors?
6. How to measure the validity and reliability of created measures?



Q1: General steps of PCA and FA

- Step 1: Selecting and Measuring a set of variables in a given domain
- Step 2: Data screening and pre-tests in order to prepare the correlation matrix
- Step 3: Factor Extraction
- Step 4: Factor Rotation to increase interpretability
- Step 5: Interpretation
- Further Steps: Validation and Reliability of the measures



Pre-test: Bartlett's Test of Sphericity

- **Bartlett Test of Sphericity** is used to test the hypothesis if the correlation matrix is an **identity matrix** (all diagonal terms are 1 and all off-diagonal terms are 0)
- H_0 : the variables are orthogonal
 H_1 : the variables are not orthogonal
- If the **value of the test** statistic for sphericity **is large** and the associated **significance level is small**, it is **unlikely** that the population correlation matrix is an identity matrix



Pre-test: Bartlett's Test of Sphericity

- If the hypothesis that the population correlation matrix is an identity cannot be rejected because the observed significance level is large, the use of the factor model should be reconsidered
- The Bartlett's test has a **strong drawback**
- It tends to be **always statistically significant** when the number of instances 'n' increases
- Some references advise to use this test only if the ratio 'n:k' (number of instances divided by the number of variables) is lower than 5



- The Kaiser-Meyer-Olkin (KMO) is a measure of sampling adequacy
- It is an index for comparing the magnitude of the observed correlation coefficients to the magnitude of the partial correlation coefficients
- The closer the KMO measure to 1 indicate a sizeable sampling adequacy (.8 and higher are great, .7 is acceptable, .6 is mediocre, less than .5 is unacceptable)
- Reasonably large values are needed for a good factor analysis. Small KMO values indicate that a factor analysis of the variables may not be a good idea



Q2: How many components/factors to retain?

- A good factor:
 - Makes sense
 - Will be easy to interpret
 - Has a simple structure
 - Lacks complex loadings
- Kaiser criterion (K1 Rule): retain only those components with an eigenvalue of greater than 1
- Kaiser criterion can lead to retaining more components than necessary!

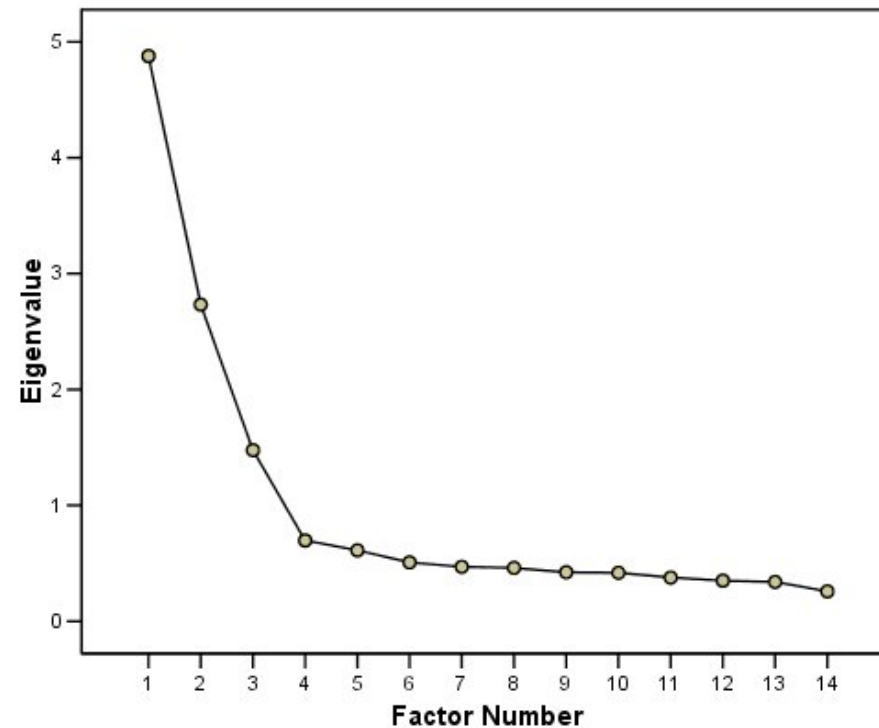


Q2: How many components/factors to retain?

- **Graphical solution - Scree Plot**
- Plots eigenvalues on Y axis and component number on X axis
- A scree plot displays the eigenvalues associated with a component or factor in descending order versus the number of the component or factor
- Recommendation is to retain all components in the descent before the first one on the line where it levels off
- The ideal pattern in a scree plot is a steep curve, followed by a bend and then a flat or horizontal line



Q2: How many components/factors to retain?



Example – Retain 3 components



Q3 - Why rotation?

- **Un-rotated** factors are typically **not very interpretable** (most factors are correlated with many variables)
- Factors are **rotated to make them more meaningful** and easier to interpret (each variable is associated with a minimal number of factors)
- **Different rotation** methods may result in the identification of somewhat **different factors**
- The most popular rotation method is **Varimax rotation**
- Other methods are Quartimax, Equamax, Promax



Q3 – How does rotation work?

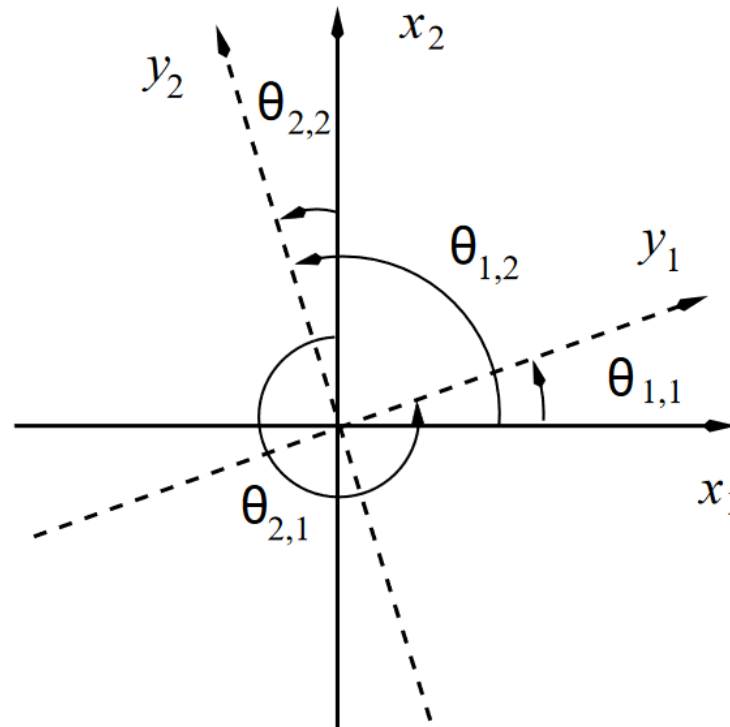


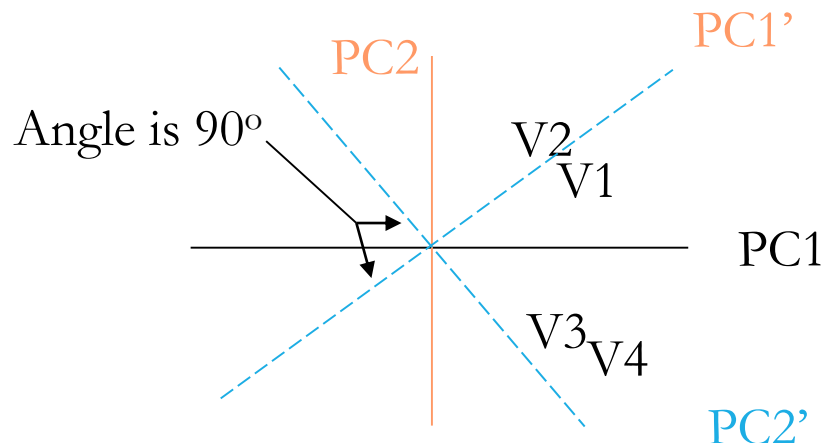
Figure 1: An orthogonal rotation in 2 dimensions. The angle of rotation between an old axis m and a new axis n is denoted by $\theta_{m,n}$.



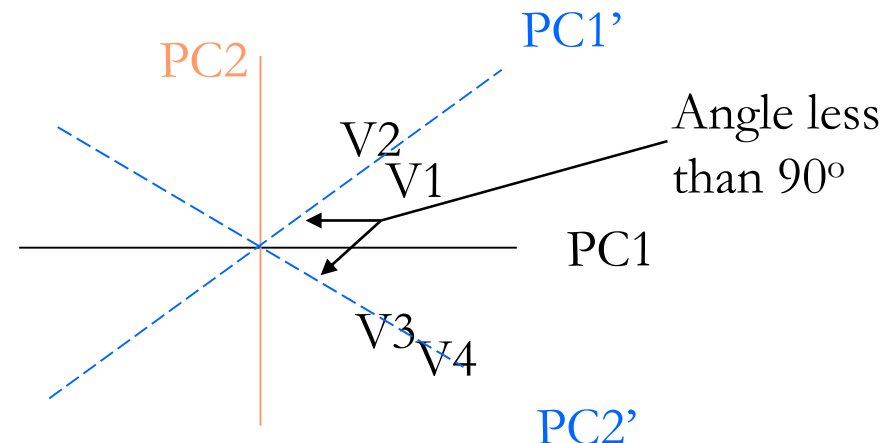
Q3 – Types of rotation

- Orthogonal Rotation - resulting factors are uncorrelated
 - more parsimonious & efficient, but less “natural”
- Oblique Rotation - resulting factors are correlated
 - more “natural” & better “spearing”, but more complicated

Orthogonal Rotation



Oblique Rotation



Q3 – Types of rotation

Orthogonal Rotations	Oblique Rotations
Varimax • most commonly used • maximizes variance of loadings of variables of a factor • tends to produce group factors	Promax • computes best orthogonal solution • “relaxes” orthogonality constraints to better “spear” variable clusters with factor vectors (give simpler structure)
Quartimax • maximizes variance of loadings of a variable across factors • tends to produce a general & small group factors	Direct Oblimin • spearing variable clusters as well as possible to produce lowest occurrence of multi-vocality
Equamax • designed to “balance” varimax and quartimax tendencies	



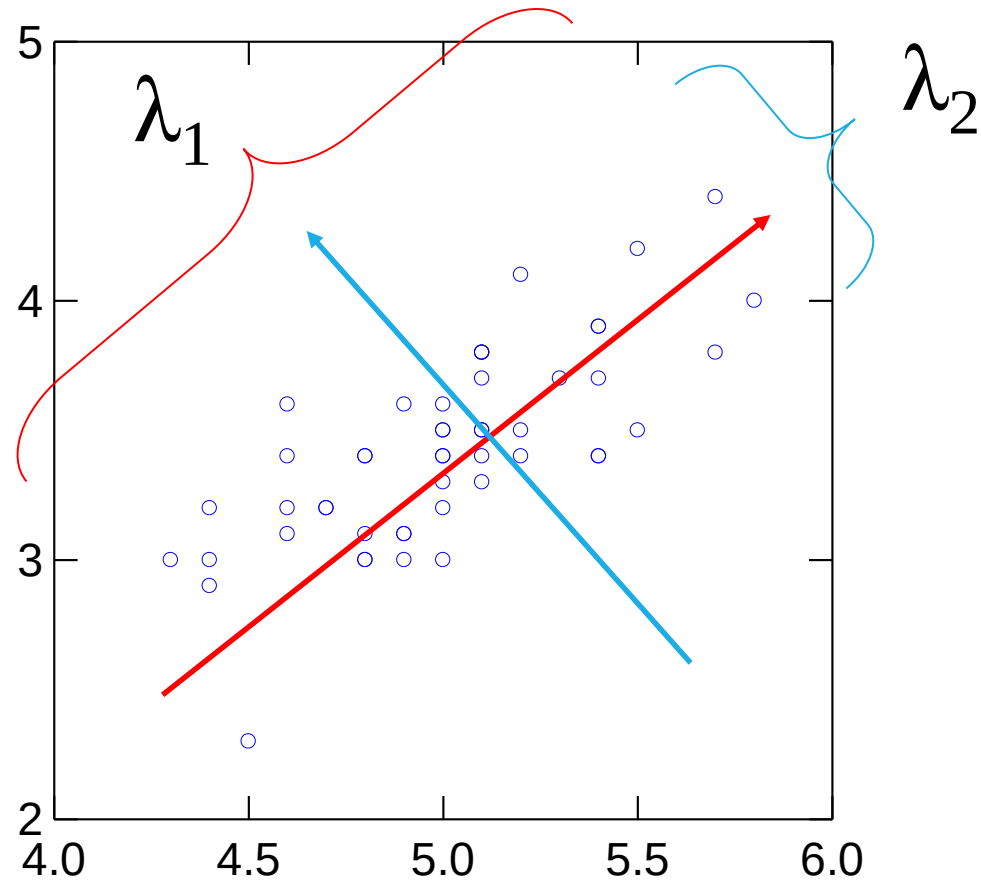
- **Communality.** This is the proportion of each variable's variance that can be explained by the factors. It is also noted as h^2 and can be defined as the sum of squared factor loadings for the variables.
- It can be:
 - **Initial** - squared multiple correlation of the variable with the other variables
 - **Extraction** - indicates the proportion of each variable's variance that can be explained by the retained factors. Variables with high values are well represented in the common factor space, while variables with low values are not well represented.



- **Eigenvalue** - Represents the total variance explained by each factor. The **first factor** will always account for **the most** variance (and hence have the highest eigenvalue), and the next factor will account for as much of the left-over variance as it can, and so on. Hence, each successive factor will account for less and less variance
- **Initial Eigenvalues - % of Variance** - This column contains the percent of total variance accounted for by each factor
- **Initial Eigenvalues - Cumulative %** - This column contains the cumulative percentage of variance accounted for by the current and all preceding factors



PCA: Eigenvalues



- **Factor loadings** - Correlations between the variables and the factors. Because these are correlations, possible values range from -1 to +1. It is usual not to report any correlations that are less than $|\cdot 3|$
- **Factor matrix** - A factor matrix contains the unrotated factor loadings of all the variables on all the factors
- **Factor scores** - Factor scores are composite scores estimated for each respondent on the derived factors



Q5: How to interpret the retained components/factors?

- Interpreting the results can be a difficult task
- Assign a name to each retained component based on the variables which make it
- To do this, we need to know what variables make the components using the component matrix



Q6 – Measure of validity and consistency

- Cronbach's alpha
- Measures to which extent scores on a test are consistent across multiple administrations of the test; the amount of measurement error in the scores yielded by a test
- Benchmarks for Alpha:
 - .9 & up = very good
 - .8 to .9 = good
 - .7 to .8 = acceptable
 - .7 & below = suspect.



- As PCA, the Factor analysis is a method of data reduction
- FA and PCA are not much different
- Factor analysis is a general name denoting a class of procedures
- It can be used to reduce the information in a model, to verify scale construction, to construct indices, ...



- **Exploratory FA**

- Summarizing data by grouping correlated variables
- Investigating sets of measured variables related to theoretical constructs
- Usually done near the onset of research

- **Confirmatory FA**

- More advanced technique
- When factor structure is known or at least theorized
- Testing generalization of factor structure to new data, etc.
- This is tested through SEM methods



Factor Analysis Model

- The first set of weights (factor score coefficients) are chosen so that the first factor explains the largest portion of the total variance.
- Then a second set of weights can be selected, so that the second factor explains most of the residual variance, subject to being uncorrelated with the first factor.
- This same principle applies for selecting additional weights for the additional factors.



```
install.packages("foreign")  
  
library(foreign)  
  
dataset = read.spss(file.choose(),  
to.data.frame=TRUE)  
  
data.use = subset(dataset, select=-  
c(Country))  
  
matrixdata<-data.matrix(data.use)
```

- Install package foreign
- Call the package foreign
- Load the data set (Food.sav)
- Remove the Country name from the dataset
- Create a matrix of the data you will use



```
install.packages("REdaS")
```

```
library(REdaS)
```

```
bart_spher(matrixdata)
```

- Install package REdaS
- Call the package REdaS
- Conduct Bartlett's test of sphericity



```
install.packages("REdaS")
```

```
library(REdaS)
```

```
bart_spher(matrixdata)
```

- Install package REdaS
- Call the package REdaS
- Conduct Bartlett's test of sphericity

Bartlett's Test of Sphericity

```
Call: bart_spher(x = matrixdata)
```

```
x2 = 1854.129
```

```
df = 171
```

```
p-value < 2.22e-16
```



```
install.packages("REdaS")
```

```
library(REdaS)
```

```
bart_spher(matrixdata)
```

```
KMOS(matrixdata)
```

- Install package REdaS
- Call the package REdaS
- Conduct Bartlett's test of sphericity
- Obtain the value of KMO measure



```
install.packages("REdaS")
```

```
library(REdaS)
```

```
bart_spher(matrixdata)
```

```
KMOS(matrixdata)
```

- Install package REdaS
- Call the package REdaS
- Conduct Bartlett's test of sphericity
- Obtain the value of the KMO statistics

Kaiser-Meyer-Olkin Statistics

Call: KMOS(x = matrixdata)

Measures of Sampling Adequacy (MSA):

v1	v2	v3	v4	v5	v6	v7	v8	v9	v10
0.9342774	0.8766698	0.9438805	0.1794681	0.9647172	0.9254329	0.9191073	0.8446491	0.9539570	0.4264564
v11	v12	v13	v14	v15	v16	v17	v18	v19	
0.8579077	0.9057275	0.4291993	0.8806570	0.9228077	0.9388467	0.9260383	0.9343596	0.9326593	

KMO-Criterion: 0.9144029



```
install.packages("car")
```

```
library(car)
```

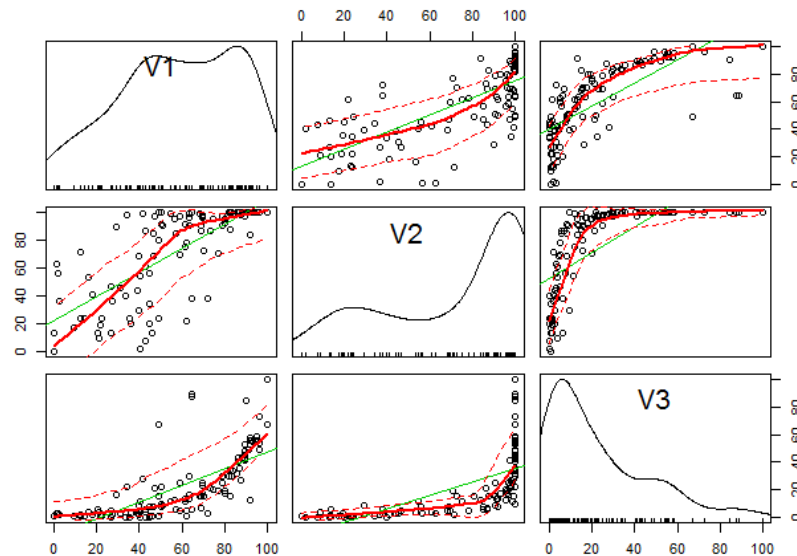
```
scatterplotMatrix(data.use[1:3])
```

- Install package car
- Call the package car
- Plot scatterplot of certain variables



```
install.packages("car")  
  
library(car)  
  
scatterplotMatrix(data.use[1:3])
```

- Install package car
- Call the package car
- Plot scatterplot of certain variables



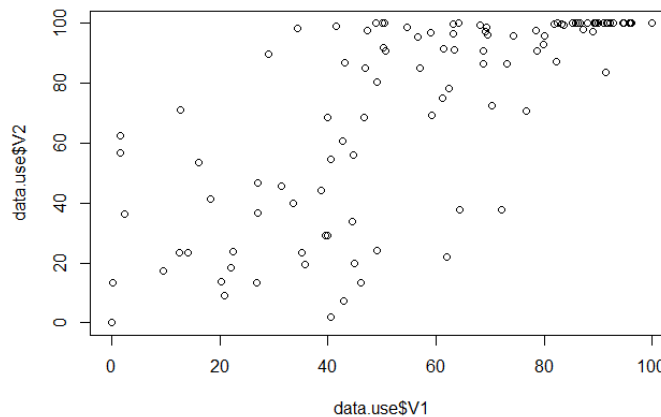
```
install.packages("car")  
  
library(car)  
  
scatterplotMatrix(data.use[1:3])  
  
plot(data.use$V1, data.use$V2)
```

- Install package car
- Call the package car
- Plot scatterplot of certain variables
- Plot scatterplot of just two variables



```
install.packages("car")  
  
library(car)  
  
scatterplotMatrix(data.use[1:3])  
  
plot(data.use$V1, data.use$V2)
```

- Install package car
- Call the package car
- Plot scatterplot of certain variables
- Plot scatterplot of just two variables



```
food.pca<-prcomp(data.use, scale. = TRUE)
```

```
summary(food.pca)
```

- Conduct the basic PCA analysis
- Get the summary of the PCA results



```
food.pca<-prcomp(data.use)
```

```
summary(food.pca)
```

- Conduct the basic PCA analysis
- Get the summary of the PCA results

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	PC11
Standard deviation	3.212	1.22837	1.05147	1.01751	0.93976	0.88509	0.81229	0.75187	0.65075	0.60344	0.54499
Proportion of Variance	0.543	0.07942	0.05819	0.05449	0.04648	0.04123	0.03473	0.02975	0.02229	0.01917	0.01563
Cumulative Proportion	0.543	0.62245	0.68064	0.73513	0.78161	0.82284	0.85757	0.88732	0.90961	0.92878	0.94441

	PC12	PC13	PC14	PC15	PC16	PC17	PC18	PC19
Standard deviation	0.46665	0.42448	0.40512	0.37624	0.34180	0.30462	0.2892	0.24358
Proportion of Variance	0.01146	0.00948	0.00864	0.00745	0.00615	0.00488	0.0044	0.00312
Cumulative Proportion	0.95587	0.96535	0.97399	0.98144	0.98759	0.99247	0.9969	1.00000



```
food.pca<-prcomp(data.use)
```

```
summary(food.pca)
```

- Conduct the basic PCA analysis
- Get the summary of the PCA results

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	PC11
Standard deviation	3.212	1.22837	1.05147	1.01751	0.93976	0.88509	0.81229	0.75187	0.65075	0.60344	0.54499
Proportion of Variance	0.543	0.07942	0.05819	0.05449	0.04648	0.04123	0.03473	0.02975	0.02229	0.01917	0.01563
Cumulative Proportion	0.543	0.62245	0.68064	0.73513	0.78161	0.82284	0.85757	0.88732	0.90961	0.92878	0.94441

	PC12	PC13	PC14	PC15	PC16	PC17	PC18	PC19
Standard deviation	0.46665	0.42448	0.40512	0.37624	0.34180	0.30462	0.2892	0.24358
Proportion of Variance	0.01146	0.00948	0.00864	0.00745	0.00615	0.00488	0.0044	0.00312
Cumulative Proportion	0.95587	0.96535	0.97399	0.98144	0.98759	0.99247	0.9969	1.00000

Proportion of the
variance explained by the
i-th component

Proportion of the
variance explained by the
i retained components

St. dev of the
components - the square
roots of the eigenvalues



```
food.pca<-prcomp(data.use)
```

```
summary(food.pca)
```

```
food.pca$sdev
```

- Conduct the basic PCA analysis
- Get the summary of the PCA results
- Shows the the square roots of the eigenvalues



```
food.pca<-prcomp(data.use)
```

```
summary(food.pca)
```

```
food.pca$sdev
```

- Conduct the basic PCA analysis
- Get the summary of the PCA results
- Shows the the square roots of the eigenvalues

```
[1] 3.2121075 1.2283721 1.0514748 1.0175130 0.9397632 0.8850909 0.8122877 0.7518726 0.6507502 0.6034433  
[11] 0.5449904 0.4666475 0.4244791 0.4051235 0.3762430 0.3418041 0.3046205 0.2892358 0.2435765
```



```
food.pca<-prcomp(data.use)
```

```
summary(food.pca)
```

```
food.pca$sdev
```

```
(food.pca$sdev)^2
```

- Conduct the basic PCA analysis
- Get the summary of the PCA results
- Shows the the square roots of the eigenvalues
- Obtain the eigenvalues



```
food.pca<-prcomp(data.use)
```

```
summary(food.pca)
```

```
food.pca$sdev
```

```
(food.pca$sdev)^2
```

- Conduct the basic PCA analysis
- Get the summary of the PCA results
- Shows the the square roots of the eigenvalues
- Obtain the eigenvalues

```
[1] 10.31763463  1.50889798  1.10559921  1.03533266  0.88315483  0.78338592  0.65981136  0.56531237  0.42347584  
[10]  0.36414388  0.29701457  0.21775989  0.18018248  0.16412506  0.14155882  0.11683002  0.09279364  0.08365733  
[19]  0.05932952
```



```
food.pca$rotation
```

- Linear combination of the all components



```
food.pca$rotation
```

- Linear combination of the all components

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
v1	0.270881854	-0.10809814	0.004489528	-0.09182336	0.034394450	-0.064849955	0.12758640	-0.1483310119
v2	0.265831087	0.06665496	-0.239339403	0.17762247	-0.192083498	-0.003160130	0.23614393	-0.0040189220
v3	0.270581084	0.05685955	0.085344952	0.01856887	0.029670525	0.092229921	-0.28616526	0.1022500050
v4	0.014684251	-0.12933493	0.524529076	0.75715130	-0.152800773	0.229096654	0.03784724	0.0005518041
v5	0.277514920	0.02908848	-0.107412403	0.07281330	-0.060364364	0.123555906	0.09237764	0.1062036657
v6	0.292900309	0.01881886	-0.114260426	0.03613691	-0.003037344	0.025742243	-0.09210852	-0.0393654087
v7	0.274337244	0.15466098	-0.149567762	-0.05130099	-0.045318441	0.198847929	0.16475962	-0.0433666486
v8	0.150211629	-0.06407133	0.500978342	-0.29445081	0.142914490	-0.194619373	0.69247704	0.1250849646
v9	0.278932024	-0.05881233	-0.003749036	-0.10582577	0.014112219	-0.084917732	-0.13275518	0.1004239800
v10	-0.003245728	-0.44348657	-0.314837957	0.21164883	0.711493807	0.289157963	0.18259534	0.1168786907
v11	0.227930540	-0.32282341	0.239534109	-0.18552430	0.051940158	0.063050443	-0.34650148	0.0632699123
v12	0.248172711	-0.13072803	0.289727260	-0.20626191	0.127952382	0.017225713	-0.30854654	0.2386422643
v13	-0.037642976	-0.58667957	-0.193110466	-0.09022679	-0.567401485	0.094563957	0.10958421	0.3917080388
v14	0.168592735	0.06769684	-0.145549898	0.35213915	0.144245255	-0.719931512	-0.09346128	0.3928487376
v15	0.261778566	0.10038295	0.117075930	0.12530655	-0.049401652	-0.001426333	0.05299615	-0.1016403350
v16	0.165875027	-0.43483677	-0.029238604	0.03809826	-0.042514733	-0.296556720	-0.06480055	-0.7291114969
v17	0.242024624	0.19303944	-0.099326521	-0.03281524	-0.080612362	0.281506034	0.01271713	0.0123489304
v18	0.276077274	0.16517092	0.006728319	-0.01203038	0.111276263	0.210742827	-0.01643826	-0.0403008509
v19	0.264318492	-0.02758150	-0.197318794	0.08354584	-0.142088115	-0.086612196	0.16289383	-0.0315515587

$$PC_1 = 0.270 \cdot V_1 + 0.265 \cdot V_2 + 0.240 \cdot V_3 + \dots + 0.264 \cdot V_{19}$$



```
food.pca$rotation
```

```
food.pca$rotation[,1]
```

- Linear combination of the all components
- Linear combination of the first component



```
food.pca$rotation
```

```
food.pca$rotation[,1]
```

- Linear combination of the all components
- Linear combination of the first component

v1	v2	v3	v4	v5	v6	v7	v8
0.270881854	0.265831087	0.270581084	0.014684251	0.277514920	0.292900309	0.274337244	0.150211629
v9	v10	v11	v12	v13	v14	v15	v16
0.278932024	-0.003245728	0.227930540	0.248172711	-0.037642976	0.168592735	0.261778566	0.165875027
v17	v18	v19					
0.242024624	0.276077274	0.264318492					



```
food.pca$rotation
```

```
food.pca$rotation[,1]
```

```
food.pca$rotation[,1:2]
```

- Linear combination of the all components
- Linear combination of the first component
- Linear combination of the 1st and 2nd component



```
food.pca$rotation
```

```
food.pca$rotation[,1]
```

```
food.pca$rotation[,1:2]
```

- Linear combination of the all components
- Linear combination of the first component
- Linear combination of the 1st and 2nd component

	PC1	PC2
v1	0.270881854	-0.10809814
v2	0.265831087	0.06665496
v3	0.270581084	0.05685955
v4	0.014684251	-0.12933493
v5	0.277514920	0.02908848
v6	0.292900309	0.01881886
v7	0.274337244	0.15466098
v8	0.150211629	-0.06407133
v9	0.278932024	-0.05881233
v10	-0.003245728	-0.44348657
v11	0.227930540	-0.32282341
v12	0.248172711	-0.13072803
v13	-0.037642976	-0.58667957
v14	0.168592735	0.06769684
v15	0.261778566	0.10038295
v16	0.165875027	-0.43483677
v17	0.242024624	0.19303944
v18	0.276077274	0.16517092
v19	0.264318492	-0.02758150



```
food.pca$rotation
```

```
food.pca$rotation[,1]
```

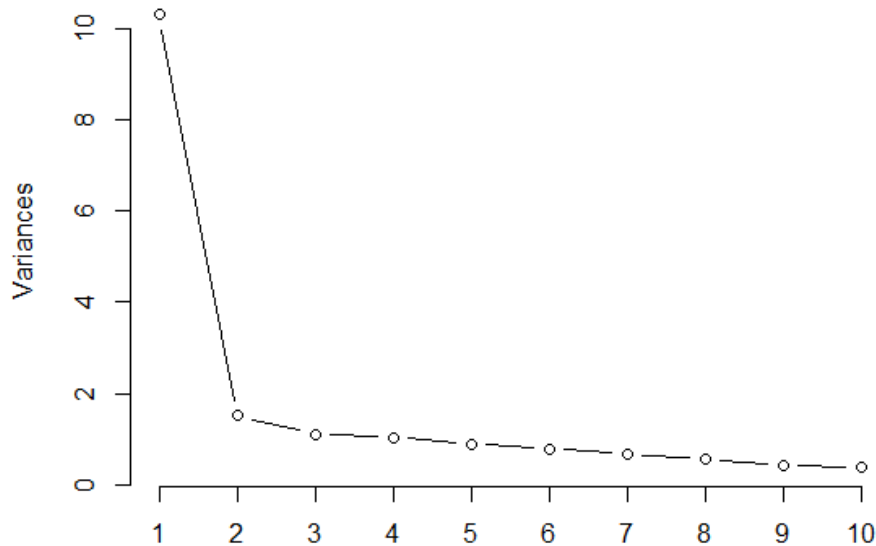
```
food.pca$rotation[,1:2]
```

```
screeplot(food.pca, main="Scree  
plot", type="lines")
```

- Linear combination of the all components
- Linear combination of the first component
- Linear combination of the 1st and 2nd component
- Draw the scree plot



Scree plot



Retain two components



PCA in R – Scatter plot of principal components

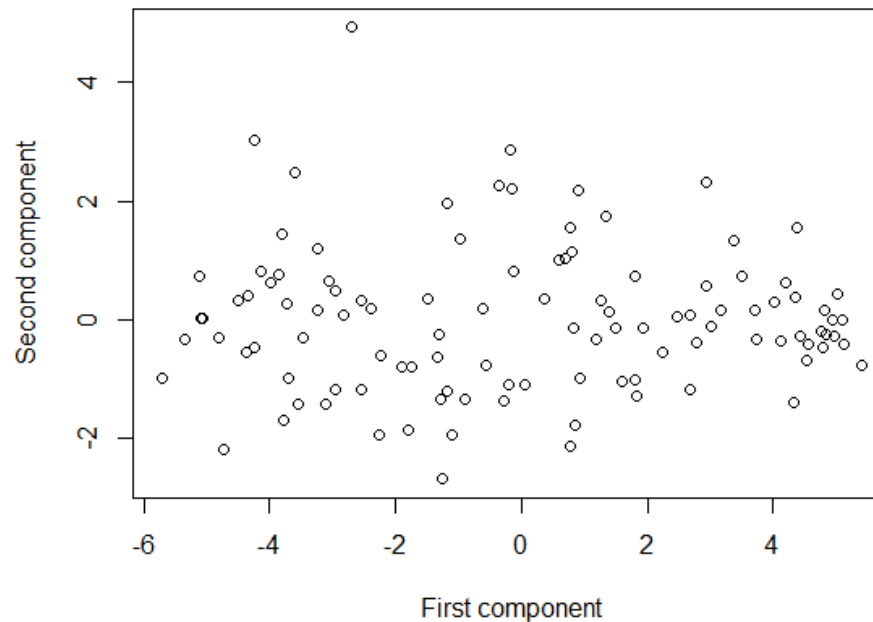
```
plot(food.pca$x[,1], food.pca$x[,2], xlab="First component", ylab="Second component", main = "Scatter plot of the first and second principal components")
```



PCA in R – Scatter plot of principal components

```
plot(food.pca$x[,1], food.pca$x[,2], xlab="First component", ylab="Second component", main = "Scatter plot of the first and second principal components")
```

Scatter plot of the first and second principal components



Factor analysis in R

```
install.packages("psych")
```

```
library(psych)
```

- Install package psych
- Call the package psych

We skip the Bartlett's test and KMO as we have done them previously



Factor analysis in R

```
install.packages("psych")  
  
library(psych)  
  
food.factor <- principal(data.use)  
  
food.factor
```

- Install package psych
- Call the package psych
- Conduct the basic Factor analysis
- Print the output



Factor analysis in R – Basic output

Principal Components Analysis

Call: `principal(r = data.use)`

Standardized loadings (pattern matrix) based upon correlation matrix

	PC1	h2	u2	com
v1	0.87	0.75708	0.24	1
v2	0.85	0.72911	0.27	1
v3	0.87	0.75540	0.24	1
v4	0.05	0.00222	1.00	1
v5	0.89	0.79461	0.21	1
v6	0.94	0.88516	0.11	1
v7	0.88	0.77651	0.22	1
v8	0.48	0.23280	0.77	1
v9	0.90	0.80274	0.20	1
v10	-0.01	0.00011	1.00	1
v11	0.73	0.53603	0.46	1
v12	0.80	0.63546	0.36	1
v13	-0.12	0.01462	0.99	1
v14	0.54	0.29326	0.71	1
v15	0.84	0.70705	0.29	1
v16	0.53	0.28388	0.72	1
v17	0.78	0.60436	0.40	1
v18	0.89	0.78640	0.21	1
v19	0.85	0.72083	0.28	1



Retains just one component

Is there just **ONE** component?



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=5, rotation="none")
```

```
food.factor
```

- Conduct the FA, but retain several factors
- Print the results

```
Principal Components Analysis  
Call: principal(r = data.use, nfactors = 5, rotate = "none")  
Standardized loadings (pattern matrix) based upon correlation matrix
```

	PC1	PC2	PC3	PC4	PC5	h2	u2	com
v1	0.87	0.13	0.00	-0.09	0.03	0.78	0.215	1.1
v2	0.85	-0.08	-0.25	0.18	-0.18	0.86	0.136	1.4
v3	0.87	-0.07	0.09	0.02	0.03	0.77	0.231	1.0
v4	0.05	0.16	0.55	0.77	-0.14	0.95	0.054	2.0
v5	0.89	-0.04	-0.11	0.07	-0.06	0.82	0.183	1.1
v6	0.94	-0.02	-0.12	0.04	0.00	0.90	0.099	1.0
v7	0.88	-0.19	-0.16	-0.05	-0.04	0.84	0.158	1.2
v8	0.48	0.08	0.53	-0.30	0.13	0.62	0.376	2.8
v9	0.90	0.07	0.00	-0.11	0.01	0.82	0.180	1.0
v10	-0.01	0.54	-0.33	0.22	0.67	0.90	0.100	2.7
v11	0.73	0.40	0.25	-0.19	0.05	0.79	0.205	2.0
v12	0.80	0.16	0.30	-0.21	0.12	0.81	0.187	1.6
v13	-0.12	0.72	-0.20	-0.09	-0.53	0.87	0.132	2.1
v14	0.54	-0.08	-0.15	0.36	0.14	0.47	0.530	2.1
v15	0.84	-0.12	0.12	0.13	-0.05	0.76	0.244	1.1
v16	0.53	0.53	-0.03	0.04	-0.04	0.57	0.427	2.0
v17	0.78	-0.24	-0.10	-0.03	-0.08	0.68	0.322	1.2
v18	0.89	-0.20	0.01	-0.01	0.10	0.84	0.161	1.1
v19	0.85	0.03	-0.21	0.09	-0.13	0.79	0.210	1.2

- Component matrix (Columns PC1-PC5)
- Communality of variable (Column h2)
- Variable's uniqueness (Column u2)



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=5, rotation="none")
```

```
food.factor
```

- Conduct the FA, but retain several factors
- Print the results

	PC1	PC2	PC3	PC4	PC5
SS loadings	10.32	1.51	1.11	1.04	0.88
Proportion Var	0.54	0.08	0.06	0.05	0.05
Cumulative Var	0.54	0.62	0.68	0.74	0.78
Proportion Explained	0.69	0.10	0.07	0.07	0.06
Cumulative Proportion	0.69	0.80	0.87	0.94	1.00

Mean item complexity = 1.6

Test of the hypothesis that 5 components are sufficient.

The root mean square of the residuals (RMSR) is 0.05
with the empirical chi square 83.41 with prob < 0.56

Fit based upon off diagonal values = 0.99



Retain just four components



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=5, rotation="none")
```

```
food.factor
```

```
vss.scree(food.factor$loadings)
```

- Conduct the FA, but retain several factors
- Print the results
- Give the Scree plot



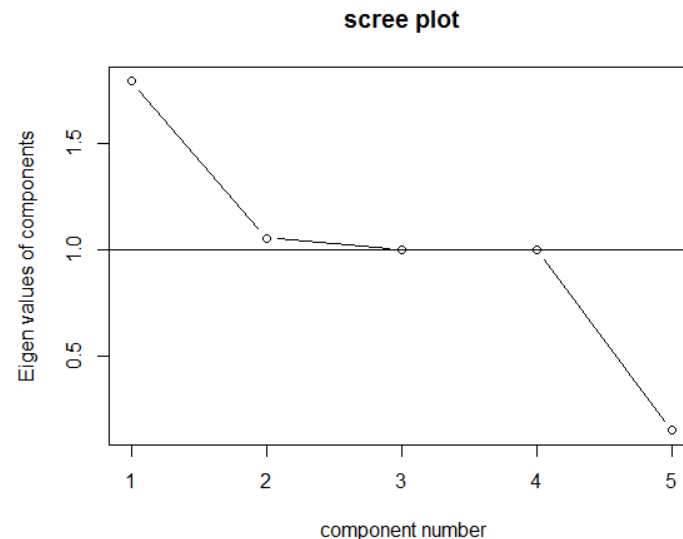
Factor analysis in R

```
food.factor <- principal(data.use,  
n factors=5, rotation="none")
```

```
food.factor
```

```
vss.scrree(food.factor$loadings)
```

- Conduct the FA, but retain several factors
- Print the results
- Give the Scree plot



```
food.factor <- principal(data.use,  
nfactors=4, rotation="varimax")
```

```
food.factor
```

- Conduct the FA, but retain several factors
- Print the results

```
Principal Components Analysis  
Call: principal(r = data.use, nfactors = 4, rotate = "varimax")  
Standardized loadings (pattern matrix) based upon correlation matrix
```

	RC1	RC3	RC2	RC4	h2	u2	com
v1	0.72	0.50	0.14	-0.04	0.78	0.217	1.9
v2	0.90	0.12	0.07	0.03	0.83	0.168	1.0
v3	0.76	0.43	-0.07	0.07	0.77	0.231	1.6
v4	-0.01	0.07	0.02	0.96	0.93	0.075	1.0
v5	0.85	0.29	0.05	0.02	0.81	0.186	1.2
v6	0.89	0.33	0.06	-0.02	0.90	0.099	1.3
v7	0.86	0.27	-0.09	-0.13	0.84	0.160	1.3
v8	0.16	0.75	-0.15	0.05	0.61	0.394	1.2
v9	0.75	0.49	0.08	-0.07	0.82	0.180	1.8
v10	0.04	-0.15	0.65	0.07	0.45	0.547	1.1
v11	0.43	0.73	0.27	0.04	0.79	0.208	1.9
v12	0.52	0.73	0.03	0.02	0.80	0.202	1.8
v13	-0.21	0.09	0.72	-0.10	0.58	0.416	1.2
v14	0.63	-0.05	0.04	0.22	0.45	0.548	1.3
v15	0.75	0.37	-0.12	0.17	0.75	0.246	1.6
v16	0.38	0.39	0.52	0.09	0.57	0.428	2.8
v17	0.77	0.23	-0.15	-0.10	0.67	0.327	1.3
v18	0.82	0.36	-0.16	-0.01	0.83	0.172	1.5
v19	0.84	0.23	0.15	-0.02	0.77	0.228	1.2



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=4, rotation="varimax")
```

```
food.factor
```

- Conduct the FA, but retain several factors
- Print the results

	RC1	RC3	RC2	RC4
SS loadings	8.31	3.14	1.45	1.07
Proportion Var	0.44	0.17	0.08	0.06
Cumulative Var	0.44	0.60	0.68	0.74
Proportion Explained	0.60	0.22	0.10	0.08
Cumulative Proportion	0.60	0.82	0.92	1.00

```
Mean item complexity = 1.5
```

```
Test of the hypothesis that 4 components are sufficient.
```

```
The root mean square of the residuals (RMSR) is 0.05  
with the empirical chi square 111.92 with prob < 0.22
```

```
Fit based upon off diagonal values = 0.99
```



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=4, rotation="varimax")
```

```
food.factor
```

```
food.factor$loadings
```

- Conduct the FA, but retain several factors
- Print the results
- Just print the loadings



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=4, rotation="varimax")
```

```
food.factor
```

```
food.factor$loadings
```

- Conduct the FA, but retain several factors
- Print the results
- Just print the loadings

```
Loadings:
      RC1    RC3    RC2    RC4
v1  0.718  0.498  0.137
v2  0.901  0.117
v3  0.756  0.433
v4                                0.959
v5  0.852  0.294
v6  0.887  0.334
v7  0.862  0.269    -0.132
v8  0.157  0.746  -0.153
v9  0.753  0.491
v10             -0.149  0.651
v11  0.432  0.731  0.266
v12  0.516  0.729
v13 -0.208             0.723
v14  0.632             0.221
v15  0.755  0.373  -0.119  0.175
v16  0.378  0.385  0.521
v17  0.766  0.231  -0.154
v18  0.821  0.358  -0.160
v19  0.835  0.228  0.150
```



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=4, rotation="varimax")
```

```
food.factor
```

```
food.factor$loadings
```

```
food.factor$values
```

- Conduct the FA, but retain several factors
- Print the results
- Just print the loadings
- Print the eigenvalues of factors



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=4, rotation="varimax")  
  
food.factor  
  
food.factor$loadings  
  
print(food.factor$loadings, cutoff=0.3)
```

- Conduct the FA, but retain several factors
- Print the results
- Just print the loadings
- Print the loadings which are more than 0.3



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=4, rotation="varimax")
```

```
food.factor
```

```
food.factor$loadings
```

```
print(food.factor$loadings, cutoff=0.3)
```

- Conduct the FA, but retain several factors
- Print the results
- Just print the loadings
- Print the loadings which are more than 0.3

```
Loadings:
      RC1    RC3    RC2    RC4
v1  0.718  0.498
v2  0.901
v3  0.756  0.433
v4                                0.959
v5  0.852
v6  0.887  0.334
v7  0.862
v8                                0.746
v9  0.753  0.491
v10                                0.651
v11  0.432  0.731
v12  0.516  0.729
v13                                0.723
v14  0.632
v15  0.755  0.373
v16  0.378  0.385  0.521
v17  0.766
v18  0.821  0.358
v19  0.835
```



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=4, rotation="varimax")  
  
food.factor  
  
food.factor$loadings  
  
print(food.factor$loadings, cutoff=0.3)  
  
food.factor$values
```

- Conduct the FA, but retain several factors
- Print the results
- Just print the loadings
- Print the loadings which are more than 0.3
- Print the eigenvalues of factors



Factor analysis in R

```
food.factor <- principal(data.use,  
nfactors=4, rotation="varimax")  
  
food.factor  
  
food.factor$loadings  
  
print(food.factor$loadings, cutoff=0.3)  
  
food.factor$values
```

- Conduct the FA, but retain several factors
- Print the results
- Just print the loadings
- Print the loadings which are more than 0.3
- Print the eigenvalues of factors

```
[1] 10.31763463  1.50889798  1.10559921  1.03533266  0.88315483  0.78338592  0.65981136  0.56531237  0.42347584  
[10]  0.36414388  0.29701457  0.21775989  0.18018248  0.16412506  0.14155882  0.11683002  0.09279364  0.08365733  
[19]  0.05932952
```



```
cor(data.use)
```

- Correlation matrix



```
cor(data.use)
```

- Correlation matrix

	v1	v2	v3	v4	v5	v6	v7	v8
v1	1.00000000	0.73410255	0.71275203	-0.015791631	0.73863208	0.823363219	0.72302041	0.48311664
v2	0.73410255	1.00000000	0.66857696	0.051087266	0.79670068	0.792956482	0.83765148	0.28638921
v3	0.71275203	0.66857696	1.00000000	0.096997270	0.75323748	0.842096044	0.73217419	0.35278739
v4	-0.01579163	0.05108727	0.09699727	1.000000000	0.06841797	0.006323547	-0.05817855	0.05912497
v5	0.73863208	0.79670068	0.75323748	0.068417969	1.000000000	0.863568929	0.82890873	0.37122951
v6	0.82336322	0.79295648	0.84209604	0.006323547	0.86356893	1.000000000	0.83300951	0.33849526
v7	0.72302041	0.83765148	0.73217419	-0.058178550	0.82890873	0.833009508	1.000000000	0.37499872
v8	0.48311664	0.28638921	0.35278739	0.059124975	0.37122951	0.338495256	0.37499872	1.000000000
v9	0.78336574	0.71860574	0.81649179	-0.026234227	0.78190285	0.823699296	0.76284604	0.42979292
v10	0.04882541	-0.01960152	-0.05375319	0.025166038	0.01699937	0.012856071	-0.04478356	-0.06250940
v11	0.66902128	0.44409234	0.62702865	0.080657435	0.58887987	0.659428408	0.52288434	0.40181110
v12	0.66834440	0.52297426	0.74264539	0.045632777	0.63857725	0.688088885	0.60177312	0.49897230
v13	-0.04608676	-0.02888618	-0.16124390	0.015809385	-0.07614325	-0.118442802	-0.17040349	-0.08186590
v14	0.40916185	0.51622246	0.45016143	0.051196231	0.45638295	0.518645299	0.36842499	0.18926015
v15	0.74761181	0.76475297	0.69952656	0.149152885	0.68888464	0.781134888	0.71690534	0.40480404
v16	0.53263142	0.41138313	0.39384315	0.075099878	0.39555176	0.493989719	0.34473342	0.25530753
v17	0.56290110	0.67061608	0.69150889	-0.025164140	0.69777101	0.736913295	0.73568558	0.30506870
v18	0.72542176	0.73434854	0.78390653	0.027993345	0.77952275	0.819259943	0.85168401	0.39603903
v19	0.72319237	0.82145065	0.64344000	0.015238392	0.80360128	0.812284945	0.76429813	0.33772862



Additional analysis

```
cor(data.use)
```

```
round(cor(data.use), 3)
```

- Correlation matrix
- Round the values of the correlation matrix to 3 decimal places



Additional analysis

```
cor(data.use)
```

```
round(cor(data.use), 3)
```

- Correlation matrix
- Round the values of the correlation matrix to 3 decimal places

	v1	v2	v3	v4	v5	v6	v7	v8	v9	v10	v11	v12	v13	v14	v15
v1	1.000	0.734	0.713	-0.016	0.739	0.823	0.723	0.483	0.783	0.049	0.669	0.668	-0.046	0.409	0.748
v2	0.734	1.000	0.669	0.051	0.797	0.793	0.838	0.286	0.719	-0.020	0.444	0.523	-0.029	0.516	0.765
v3	0.713	0.669	1.000	0.097	0.753	0.842	0.732	0.353	0.816	-0.054	0.627	0.743	-0.161	0.450	0.700
v4	-0.016	0.051	0.097	1.000	0.068	0.006	-0.058	0.059	-0.026	0.025	0.081	0.046	0.016	0.051	0.149
v5	0.739	0.797	0.753	0.068	1.000	0.864	0.829	0.371	0.782	0.017	0.589	0.639	-0.076	0.456	0.689
v6	0.823	0.793	0.842	0.006	0.864	1.000	0.833	0.338	0.824	0.013	0.659	0.688	-0.118	0.519	0.781
v7	0.723	0.838	0.732	-0.058	0.829	0.833	1.000	0.375	0.763	-0.045	0.523	0.602	-0.170	0.368	0.717
v8	0.483	0.286	0.353	0.059	0.371	0.338	0.375	1.000	0.430	-0.063	0.402	0.499	-0.082	0.189	0.405
v9	0.783	0.719	0.816	-0.026	0.782	0.824	0.763	0.430	1.000	-0.013	0.689	0.764	-0.060	0.492	0.656
v10	0.049	-0.020	-0.054	0.025	0.017	0.013	-0.045	-0.063	-0.013	1.000	0.088	0.000	0.150	0.019	-0.098
v11	0.669	0.444	0.627	0.081	0.589	0.659	0.523	0.402	0.689	0.088	1.000	0.818	0.103	0.267	0.586
v12	0.668	0.523	0.743	0.046	0.639	0.688	0.602	0.499	0.764	0.000	0.818	1.000	-0.061	0.372	0.638
v13	-0.046	-0.029	-0.161	0.016	-0.076	-0.118	-0.170	-0.082	-0.060	0.150	0.103	-0.061	1.000	-0.153	-0.194
v14	0.409	0.516	0.450	0.051	0.456	0.519	0.368	0.189	0.492	0.019	0.267	0.372	-0.153	1.000	0.460
v15	0.748	0.765	0.700	0.149	0.689	0.781	0.717	0.405	0.656	-0.098	0.586	0.638	-0.194	0.460	1.000
v16	0.533	0.411	0.394	0.075	0.396	0.494	0.345	0.255	0.494	0.155	0.519	0.416	0.174	0.291	0.385
v17	0.563	0.671	0.692	-0.025	0.698	0.737	0.736	0.305	0.621	-0.083	0.436	0.563	-0.143	0.339	0.651
v18	0.725	0.734	0.784	0.028	0.780	0.819	0.852	0.396	0.761	-0.018	0.581	0.684	-0.269	0.413	0.750
v19	0.723	0.821	0.643	0.015	0.804	0.812	0.764	0.338	0.746	0.001	0.576	0.554	-0.027	0.498	0.666



Adittional analysis

```
food.factor <- principal(data.use,  
nfactors=4, scores = TRUE,  
rotate="varimax")
```

```
food.factor$scores
```

- Conduct the FA and save the factor scores
- Display the factor scores



```
food.factor <- principal(data.use,
nfactors=4, scores = TRUE,
rotate="varimax")
```

```
food.factor$scores
```

- Conduct the FA and save the factor scores
- Display the factor scores

	RC1	RC3	RC2	RC4
1	0.093558826	-0.841249565	-1.32556027	-0.78304096
2	-1.483810370	0.202771059	-1.04966151	0.56604342
3	0.908300453	-0.528986838	-0.34282614	0.70025679
4	0.942846263	1.352012937	-0.27443091	1.43973751
5	0.995425252	1.327007035	-0.05073896	0.24613758
6	0.678987895	-1.916932783	-0.39489439	0.15566326
7	-1.010912307	-0.715586922	1.52581272	0.04684936
8	1.118313893	-1.442650691	-1.19217580	0.25932145
9	0.828796883	1.467917585	-1.72596946	0.09883664
10	-1.085273398	-0.059878554	-0.31041173	-0.40283114
11	-0.304234868	-0.389620174	1.17598087	0.64622935
12	-1.371016015	3.045782018	0.39592863	0.70063516
13	0.144762126	0.724612253	-0.12978963	0.30641958
14	0.082634637	0.312791946	-0.98048918	-0.13367982
15	-0.881713871	-0.788421120	-1.24243602	0.32793087
16	-1.951156300	1.116404307	-3.53963224	-0.06647836
17	-1.128840826	-0.809961400	0.54823478	-0.14879424
18	-0.762784037	-0.465549965	-0.32063820	-0.90745382
19	0.921799228	1.441427060	0.11187333	-0.02961952
20	-1.405949041	-0.679639049	-0.66043528	-0.51083147
21	0.460905907	0.820447853	0.75408254	0.80791485
22	0.785878124	-0.964661364	1.45018380	-0.29216420
23	0.127343594	-0.304107584	1.05992938	0.36439335
24	-1.774175958	-0.529940800	0.55675939	0.79920598
25	0.064837733	0.876843057	0.56639656	0.42243819



- Used packages: foreign, REdaS, car, psych

<https://cran.r-project.org/package=foreign>

<https://cran.r-project.org/package=REdaS>

<https://cran.r-project.org/package=car>

<https://cran.r-project.org/package=psych>



Dimension reduction